

Relazione - Versione 7: Modello Linguistico GRU

Introduzione

La Versione 7 estende il modello neurale di linguaggio a caratteri ricorrente in TensorFlow della Versione 6 introducendo una gated recurrent unit (GRU).

Corpus, vocabolario, finestre di contesto di quattro caratteri, embedding appresi, stato nascosto a 16 dimensioni, split di training e validation, mini-batch mescolati, obiettivo cross-entropy, differenziazione automatica e campionamento autoregressivo restano invariati. Invece di aggiornare lo stato nascosto con una sola trasformazione tanh, il modello apprende ora un update gate, un reset gate e uno stato candidato.

```
Caratteri
↓
Identificatori interi
↓
Contesto di 4 caratteri
↓
Embedding addestrabili
↓
Stati nascosti GRU con gate
↓
Pesi di output
↓
Testo generato
```

Obiettivo della Versione 7

Il modello deve essere in grado di:

- riutilizzare lo stesso corpus di 440 caratteri, il vocabolario di 27 caratteri e le finestre di contesto di quattro caratteri della Versione 6;
- riutilizzare embedding addestrabili a 8 dimensioni e uno stato nascosto ricorrente a 16 dimensioni;
- sostituire il semplice aggiornamento ricorrente con una gated recurrent unit esplicita;
- apprendere i pesi di update gate, reset gate, stato candidato, embedding e output con `tf.GradientTape`;
- mantenere training mini-batch, monitoraggio della validation e generazione autoregressiva riproducibile;
- verificare forme della GRU, intervalli dei gate, gradienti, aggiornamenti dei parametri, retraining deterministico e generazione.

Architettura

Il flusso completo della Versione 7 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
436 esempi contesto-target da 4 caratteri
↓
Tensore input intero: (436, 4)
↓
Matrice di embedding: (27, 8)
↓
Elaborazione GRU sequenziale
↓
Update gate + reset gate + stato candidato
↓
Stato nascosto finale: 16 valori
↓
Matrice pesi di output: (16, 27)
↓
Split esempi 80 / 20 riproducibile
↓
Aggiornamenti mini-batch con GradientTape
↓
Generatore autoregressivo a finestra scorrevole
```

Il contesto contiene ancora quattro caratteri precedenti. La Versione 7 elabora i loro embedding da sinistra a destra e riutilizza gli stessi pesi GRU in ogni posizione. L'update gate controlla quanta parte dello stato nascosto precedente viene preservata, mentre il reset gate controlla come l'informazione precedente contribuisce allo stato candidato.

Finestre di contesto, split e mini-batch

Le finestre scorrevoli creano gli stessi 436 esempi input-target della Versione 6. Ogni esempio contiene quattro identificatori di carattere ordinati. Lo stato nascosto parte da zero per ogni esempio di contesto e viene aggiornato una volta per ciascun carattere dalla GRU.

```
esempi training = 348
esempi validation = 88
dimensione batch = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e usati per aggiornare i parametri. Gli esempi di validation misurano la loss senza modificare il modello. Le finestre di contesto vicine continuano a sovrapporsi, quindi lo split casuale a livello di esempio mantiene la stessa limitazione intenzionale del piccolo dataset della Versione 6.

Embedding addestrabili e pesi ricorrenti con gate

Ogni identificatore di carattere seleziona una riga dalla matrice di embedding addestrabile. I quattro vettori selezionati vengono elaborati in sequenza. In ogni posizione, l'embedding corrente e lo stato nascosto precedente vengono usati per calcolare un update gate, un reset gate e uno stato nascosto candidato.

```
Forma tensore input: (436, 4)
Forma matrice embedding: (27, 8)
Forma contesto embedded: (1, 4, 8)
Forma pesi input update: (8, 16)
Forma pesi ricorrenti update: (16, 16)
Forma pesi input reset: (8, 16)
Forma pesi ricorrenti reset: (16, 16)
Forma pesi input candidato: (8, 16)
Forma pesi ricorrenti candidato: (16, 16)
Forma stato nascosto finale: (1, 16)
Forma matrice pesi output: (16, 27)
Parametri addestrabili: 1800
```

La matrice di embedding contiene 216 valori. Ognuna delle tre trasformazioni della GRU contiene 128 pesi di input e 256 pesi ricorrenti, mentre la matrice di output contiene 432 valori. Il modello ha quindi 1800 parametri addestrabili, rispetto ai 1032 della Versione 6.

```
z_t = sigmoid(x_t W_z + h_(t-1) U_z)
r_t = sigmoid(x_t W_r + h_(t-1) U_r)
candidate_t = tanh(x_t W_h + (r_t * h_(t-1)) U_h)
h_t = z_t * h_(t-1) + (1 - z_t) * candidate_t
```

Loss stabile in TensorFlow

Gli identificatori del contesto vengono mappati negli embedding ed elaborati dalla GRU. Lo stato nascosto finale viene moltiplicato per i pesi di output per produrre i logits del carattere successivo. La sparse softmax cross-entropy misura l'errore di previsione direttamente dai logits e dagli ID target interi.

```
_, logits = gru_forward(..., inputs)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

Le loss iniziali di training e validation sono entrambe circa 3.2958, coerenti con una previsione quasi uniforme sul vocabolario di 27 caratteri.

Esecuzione controllata e riproducibilità

Il notebook mantiene l'esecuzione su CPU e sopprime i log TensorFlow di basso livello. Seed espliciti controllano l'inizializzazione dei parametri, split NumPy, shuffle dei mini-batch e campionamento del testo. Le esecuzioni ripetute sono deterministiche nello stesso ambiente software.

Differenziazione automatica

Per ogni mini-batch, `tf.GradientTape` registra i calcoli forward ricorrenti con gate e deriva la loss rispetto a tutte e otto le matrici di parametri addestrabili:

- `tf.GradientTape` registra lookup degli embedding, update e reset gate, stati candidati, aggiornamenti dello stato nascosto, proiezione di output e loss;
- `tape.gradient` calcola i gradienti per embedding, sei matrici di pesi GRU e pesi di output;
- il gradiente sparso degli embedding viene convertito in un tensore denso per questo piccolo vocabolario;
- `assign_sub` applica aggiornamenti di gradient descent a tutte e otto le variabili;
- le loss di training e validation vengono registrate dai parametri correnti a ogni epoca.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(...)
    gradients = tape.gradient(batch_loss, trainable_variables)
```

Configurazione training:

```
Learning rate: 1.0
Dimensione batch: 32
Epoche: 100
Seed: 42
```

Risultato del training

```
Loss training iniziale: 3.295838
Loss training finale: 2.659556
Loss validation iniziale: 3.295836
Loss validation finale: 2.913941
Migliore loss validation: 2.899463
Migliore epoca validation: 99
```

La GRU apprende lentamente nella prima parte del training. Le loss restano quasi invariate fino a circa 50 epoche, poi diminuiscono più chiaramente dopo circa l'epoca 60. La loss di training continua a migliorare fino all'epoca 100, mentre la loss di validation raggiunge il minimo all'epoca 99.

La loss di validation aumenta leggermente all'epoca finale dopo aver raggiunto il minimo. Questo suggerisce l'inizio di un lieve overfitting. Il notebook registra l'epoca migliore per l'analisi, ma conserva intenzionalmente i parametri finali dell'epoca 100 per la generazione.

Controlli su gradienti, gate e parametri

I test verificano gradienti finiti per tutte e otto le matrici di parametri addestrabili, forme dei gradienti corrispondenti, stati nascosti finiti, intervalli validi dei gate e che ogni matrice di parametri cambi effettivamente durante il training.

```
0 <= update_gate <= 1
0 <= reset_gate <= 1
-1 <= candidate_state <= 1
-1 <= hidden_state <= 1
assert all(gradient is not None for gradient in gradients)
assert np.max(abs(initial - trained)) > 0.0
```

Probabilità apprese

Dopo il training, il modello produce una distribuzione di probabilità del carattere successivo a partire da un contesto ordinato di quattro caratteri. Per il contesto mode, le probabilità più alte sono:

```
spazio: 0.2329 e: 0.0918 s: 0.0825
t: 0.0765 n: 0.0706 i: 0.0636
```

Generazione neurale del testo

Per ogni previsione, la finestra corrente di quattro caratteri viene mappata negli embedding ed elaborata da sinistra a destra dalla GRU. Lo stato nascosto parte da zero per quella finestra, lo stato finale produce i logits e un generatore NumPy locale campiona il carattere successivo. Il carattere più vecchio del contesto viene quindi rimosso e la finestra avanza.

```
mode -> carattere campionato
ode? -> carattere successivo campionato
...
```

Cinque campioni dal contesto addestrato mode con seed 42 sono r, e, s, o e spazio. La dimostrazione genera poi 300 caratteri dai parametri finali e lo stesso seed riproduce lo stesso testo.

Controlli inclusi

Le asserzioni del notebook verificano costruzione delle finestre di contesto, tipi di tensori e variabili, forme di tutti i parametri GRU, intervalli di gate e stato nascosto, split dei dati, loss finite, probabilità normalizzate, gradienti finiti, modifiche dei parametri, retraining deterministico, generazione riproducibile, lunghezza di output richiesta e contesto iniziale.

Un'esecuzione completa dall'inizio alla fine termina con il messaggio All checks passed.

Cosa insegna la Versione 7

La progressione centrale è:

contesto -> ID -> embedding -> gate GRU -> stato nascosto -> logits
-> loss -> gradienti -> aggiorna -> scorri -> campiona

La Versione 7 introduce la ricorrenza con gate. Il focus didattico passa dal singolo aggiornamento tanh della Versione 6 a meccanismi appresi che controllano come l'informazione precedente viene preservata e come l'informazione candidata entra nello stato nascosto.

Interpretazione

I quattro embedding del contesto vengono ancora elaborati in sequenza. L'update gate interpola tra lo stato nascosto precedente e lo stato candidato, mentre il reset gate controlla come l'informazione precedente contribuisce al calcolo del candidato. Lo stato nascosto finale a 16 dimensioni riassume il contesto ordinato prima della proiezione di output.

Il modello ha 1800 parametri perché la GRU apprende tre trasformazioni ricorrenti invece di una. La migliore loss di validation è 2.8995 all'epoca 99. Questa versione è progettata per mostrare la ricorrenza con gate, non per sostenere che un maggior numero di parametri debba migliorare le prestazioni di validation su questo piccolo corpus.

Limitazioni

- il contesto mantiene una lunghezza fissa di quattro caratteri;
- lo stato nascosto parte da zero per ogni esempio di training;
- la generazione ricalcola ogni finestra di quattro caratteri da uno stato nascosto iniziale nullo invece di preservarlo indefinitamente;
- il corpus è piccolo ed è incorporato direttamente nel notebook;
- finestre di contesto sovrapposte possono comparire tra lo split casuale di training e validation;
- la GRU non usa intenzionalmente termini di bias;
- il training usa semplice gradient descent mini-batch con learning rate fisso;
- non sono presenti early stopping o ripristino di checkpoint, quindi la generazione usa i parametri dell'epoca 100 anche se la validation è migliore all'epoca 99;
- il testo generato cattura pattern sequenziali locali ma non possiede ancora coerenza a lungo termine.

Conclusione

La Versione 7 sostituisce il semplice aggiornamento ricorrente della Versione 6 con una GRU esplicita. Gli ID dei caratteri selezionano embedding addestrabili, la GRU li elabora da sinistra a destra attraverso update e reset gate e lo stato nascosto finale predice il carattere successivo.

L'implementazione resta ispezionabile mentre introduce un controllo appreso sul flusso di informazione ricorrente. Completa la fase ricorrente con gate e prepara la versione successiva per i meccanismi di attention.